Problem A. Colorful Segments

Sort the segments by their left endpoint and decide, from left to right, whether each segment is selected. The only information that has to be remembered about the already selected segments is the maximum right endpoint among the selected segments and the color of the segment attaining that maximum.

Let

$$dp(r, c)$$

be the number of ways whose current maximum right endpoint is r and whose last color is c. Suppose the next segment is (l, r, c). This segment can be appended after every state with the same color and endpoint at most r, and after every state with the other color whose endpoint is at most $l - 1$. Therefore these states contribute to $dp(r, c)$:

$$\sum_{x \le r} dp(x, c) + \sum_{x < l} dp(x, 1 - c).$$

For states with the same color and endpoint greater than r, selecting the new segment does not change the state, so their number is simply multiplied by 2.

Maintain two lazy segment trees, one for each color, indexed by the current right endpoint. Each segment requires a constant number of range-sum queries, range multiplications by 2, and point or range additions. The empty selection can be represented by a dummy endpoint 0.

Time complexity. $O(n \log n)$ time and $O(n)$ memory.

Problem B. Be Careful 2

Use inclusion-exclusion over the set of forbidden points. For a fixed chosen subset of points, the number of squares containing all of them is determined only by the minimum axis-parallel rectangle containing the subset.

A direct inclusion-exclusion over all subsets costs $O(2^k)$. Instead, enumerate possible minimum rectangles. The sides of such a rectangle are determined by coordinates of forbidden points, so there are initially $O(k^4)$ candidates. Moreover, if a rectangle has a forbidden point strictly inside it, then the two choices "take this interior point" and "do not take it" give the same minimum rectangle with opposite signs, hence the inclusion-exclusion coefficient of that rectangle is 0.

After breaking ties so that all x- and y-coordinates are distinct, fix the left and right boundary points. Among rectangles with nonzero coefficient, there are at most four choices for the bottom and top boundaries. Thus only $O(k^2)$ rectangles contribute.

For each contributing rectangle, enumerate the side length d of the square. The contribution as a function of d is a piecewise polynomial of degree 4: the number of possible square positions is defined by linear boundary conditions, and multiplying the horizontal and vertical choices gives a quartic expression on each interval. Split at the breakpoints and recover the polynomial on every piece by interpolation.

Time complexity. $O(k^2)$ time, with linear or quadratic memory depending on the implementation.

Problem C. Connected Intervals

For an interval $[l, r]$, let $e(l, r)$ be the number of edges completely contained in the interval. In this graph, every interval satisfies

$$e(l, r) \le r - l.$$

Therefore $[l, r]$ is connected if and only if

$$e(l, r) = r - l.$$

Sweep the right endpoint r from left to right. For every possible left endpoint l, maintain

$$val_l = e(l, r) - (r - l).$$

All values are always non-positive. Hence the number of connected intervals ending at r is exactly the number of positions l where the maximum value is 0.

Use a segment tree over l. When r increases by one, update the base term $-(r - l)$ for all active l. For every new edge (u, r), this edge is contained in exactly the intervals with $l \le u$, so add 1 to the range $[1, u]$. The segment tree stores the maximum value and the number of positions attaining it.

Time complexity. $O(n \log n)$ time and $O(n)$ memory.

Problem D. DS Team Selection 2

Assume that operation type 2 has been applied k times so far. In shifted coordinates, operation type 1 changes

$$a_i \leftarrow \min(a_i, v - ki),$$

and a sum query of type 3 is answered by adding back the corresponding ki terms.

First consider the easier case where every initial value is $+\infty$. Each type 1 operation adds an upper-bounding line $v - kx$. Since k is nondecreasing over time, the new line can only affect a suffix of the current lower envelope. Thus the envelope can be maintained with a stack. A segment tree stores the actual values on intervals, each of which is an arithmetic progression, and answers range sums.

With finite initial values, each position i stays equal to its initial value until the first time some operation cuts it. After that time, it behaves exactly as in the $+\infty$ case. Online maintenance would require querying the maximum of $a_i + ki$ among all still-unaffected positions, which is a dynamic convex-hull problem. Instead, find the first affected time of every position offline by parallel binary search. The predicate is monotone: once a position is cut by time t, it is also cut by any later time.

After these first-cut times are known, process all operations once more with the stack envelope and the segment tree.

Time complexity. $O((n + q) \log(n + q))$ time and $O(n + q)$ memory.

Problem E. Computational Geometry

For fixed vertices b and c, the optimal choice of a is the vertex farthest from the line bc, because the area of triangle abc is proportional to this distance. Along a convex polygon, this distance is unimodal.

When b and c move around the polygon in the required manner, the maximizing vertex a also moves monotonically. Therefore we can use rotating calipers: maintain a pointer for a, and while moving it to the next vertex does not decrease the doubled area

$$|(b - a) \times (c - a)|,$$

advance it. Over the whole sweep, the pointer makes only a constant number of full turns.

Time complexity. $O(n)$ time and $O(1)$ extra memory.

Problem F. Puzzle: Sashigane

Think of one operation as choosing a corner of the currently remaining rectangle and covering the two boundary sides adjacent to that corner by one L-shaped piece. After removing those two sides, the remaining board is a rectangle whose two side lengths are both smaller by one.

Thus an $n \times n$ board can be reduced as

$$(n, n) \to (n-1, n-1) \to (n-2, n-2) \to \cdots \to (1, 1).$$

Maintain the four borders of the remaining rectangle. At each step, choose a corner whose removal does not delete the cell that should remain until the end, output the corresponding L-shape, and shrink the two borders. After $n-1$ steps only one cell remains.

The produced L-shapes are disjoint because each step removes one current boundary row and one current boundary column. Their union covers every cell except the final cell.

Time complexity. $O(n)$ time.

Problem G. Gem Island 2

Place the balls of the same box consecutively. Each operation chooses one existing ball uniformly and inserts a new ball immediately after it in the same box. After d operations, all positive compositions

$$x_i \geq 1, \qquad \sum_{i=1}^{n} x_i = n + d$$

are equally likely. Hence the probability problem becomes a counting problem over the

$$\binom{n-1+d}{n-1}$$

possible compositions.

The i-th largest value can be written as

$$\sum_{t \geq 1} [\text{at least } i \text{ boxes have size at least } t].$$

If we require a fixed set of k boxes to be at least t, then after reserving $t-1$ extra balls for each of them, the number of compositions is

$$f(k, t) = \binom{n-1+d-k(t-1)}{n-1}.$$

By binomial inclusion-exclusion, define

$$g(k) = \binom{n}{k} \sum_{t \geq 1} f(k, t).$$

Then the numerator for the sum of the largest r values is

$$\sum_{i=1}^{r} \sum_{k=i}^{n} \sum_{j=k}^{n} (-1)^{j-k} \binom{j}{k} g(j).$$

For a fixed j, the coefficient of $g(j)$ in this expression simplifies to a closed form involving binomial coefficients, so once all $g(j)$ are known, the final answer is obtained in $O(n)$ time.

Computing all $g(j)$ naively gives a harmonic $O(d \log d)$ sum. It can be improved by applying a Dirichlet suffix-sum transform to

$$h(x) = \binom{n-1+d-x}{n-1},$$

which computes the required sums in $O(d \log \log d)$. Finally divide by the total number of compositions, i.e. multiply by the modular inverse of $\binom{n-1+d}{n-1}$.

Time complexity. $O(n + d \log \log d)$ time and $O(n + d)$ memory.

Problem H. Not Another Path Query Problem

For a query asking whether there is a path whose bitwise AND is at least V, decompose the comparison by the first bit where the path value becomes greater than V.

For a mask x, a path has AND containing all bits of x if and only if every edge w on the path satisfies

$$(w \ \& \ x) = x.$$

Thus the condition is just connectivity in the subgraph consisting of edges that contain mask x.

For each query, create the following candidate masks. First include $x = V$. Also, for every zero bit i of V, keep all higher bits equal to V, set bit i to 1, and set all lower bits to 0. If the endpoints are connected for any of these masks, then there is a path whose AND is at least V; otherwise there is not.

Group equal masks offline. For each group, build a DSU from all edges satisfying $(w \ \& \ x) = x$, or process the groups by bit levels to avoid rebuilding from scratch too often.

Time complexity. $O((n + m + q) \log V)$ time and $O(n + m + q)$ memory.

Problem I. Heap

Process insertions in reverse order. During the original insertion of the i-th value, the new element starts at position i and moves only along the path from i to the root. Therefore, in the reverse process, the value v_i must be the first occurrence of v_i on this path in the current heap state.

Maintain the current position of every value. For $i = n, n-1, \ldots, 1$, find the current position of v_i. It must lie on the path from i to the root. Push this value back down to position i by reversing the swaps of the insertion; update all affected positions in the value-to-position table.

After this rollback, the heap is exactly the state around the i-th insertion. Check whether bit 0 or bit 1 is valid under the condition of the problem, and choose the required one. If a lexicographically smallest bit string is needed, test 0 first and choose 1 only when 0 is impossible.

Time complexity. $O(n \log n)$ time and $O(n)$ memory.

Problem J. Triangle City

Suppose we decide which edges will be used. An Euler trail exists exactly when the graph is connected, the start and end vertices have odd degree, and all other vertices have even degree.

Initially all degrees are even. Therefore we want to delete one path from the start vertex to the end vertex while keeping the graph connected. The remaining graph will then have precisely the two required odd-degree vertices.

Take a shortest path from the start to the end. Because every triangle satisfies the triangle inequality, a shortest path never needs to use two edges of the same triangle: those two edges can be replaced by the third edge without increasing the length. Hence after deleting the shortest path, every triangle still has at least two edges remaining, so the graph remains connected.

Delete the edges of this shortest path and run Hierholzer's algorithm on the remaining graph to construct the Euler trail. Restore the output to the format required by the statement.

Time complexity. $O(n^2 \log n)$ time with a standard shortest-path implementation.

Problem K. Are you a bot?

The source article used for this reconstruction does not contain an editorial section for Problem K. It goes from Problem J directly to Problem L. Therefore this section is intentionally left as a source-missing note rather than a guessed solution.

Problem L. Difficult Constructive Problem

Split the string by positions whose values are already fixed. The remaining maximal unknown intervals are independent: after the boundary values of an interval are known, choices inside the interval do not affect any other interval.

For each interval, compute the set of total contributions it can make. If the interval is in the middle, both endpoints are fixed, so the parity of its contribution is fixed; its possible values are all numbers of one parity inside some interval. If it is at the left or right end, then only one boundary, or no boundary, is fixed, and the possible values form a whole consecutive interval.

Subtract the contribution of the already fixed positions from the target. Feasibility is now a sum-of-intervals problem with possible parity restrictions. These restrictions are simple enough that after fixing a prefix choice, suffix feasibility can be checked in $O(1)$ from the remaining minimum, maximum, and parity information.

Construct the answer greedily from left to right. Try the smaller value first; if the suffix can still realize the remaining target, keep it. Otherwise choose the next possible value. Since every step preserves suffix feasibility, the process ends with a valid construction whenever one exists.

Time complexity. $O(n)$ time and $O(n)$ memory, or $O(1)$ extra memory with careful streaming of interval data.

Problem M. Trie

For a rooted tree, the sequence represented by a subtree consists of the empty string if the root is a key node, followed by the sequences of all child subtrees after prepending one character to every string. The child subtrees may be ordered by assigning letters.

To minimize the whole sequence lexicographically, assign the smallest letter to the lexicographically smallest child sequence, the next smallest letter to the next one, and so on. One subtle point is the prefix case: if sequence x is a prefix of sequence y, then x should be placed after y in the concatenation order.

One implementation is to store subtree sequences explicitly as deques and merge smaller deques into larger ones. Sort child sequences with the prefix rule above, prepend the chosen character, and merge.

A simpler implementation avoids materializing strings. Process vertices by decreasing depth. For all vertices at the same depth, build a tuple consisting of the key-node flag and the sorted list of ranks of its child sequences. Sort these tuples and assign equal tuples the same new rank. Once all children of a vertex have ranks, this tuple fully determines the relative order of its subtree sequence.

This rank compression is equivalent to comparing the explicit sequences, because the comparison of a parent depends only on the relative order of its child sequences and on whether the empty string is present.

Time complexity. $O(n \log n)$ time and $O(n)$ memory.

Source reconstructed from the Luogu editorial article by jiangly. The linked source contains solutions for A-J, L, and M; Problem K is recorded as missing.